Opdracht 1 Zij a $\in]0,1[$ een reële parameter, gedefinieerd zoals in de opgave. Dan zoeken we een functie F, met een vast punt x^* zodat de convergentiefactor in x^* gelijk is aan a.
Beschouw hiertoe de functie $f : \mathbb{R} \to \mathbb{R} : x \mapsto ax$. Mogelijke vaste punten zijn $a = 1$, hetgeen onmogelijk is wegens de definitie van a, of $x^* = 0$.
We berekenen de convergentiefactor in het punt $x^* = 0$. Definieer daartoe de rij $(x_k)_k$ als $(x_{k+1}) = a \cdot x_k$. Dan is de convergentiefactor $\rho = \frac{|\epsilon^{(k)}|}{|\epsilon^{(k-1)}|} = \frac{|x^* - x_k|}{|x^* - x_{k-1}|} = \frac{|x_k|}{|x_{k-1}|} = \frac{|a * x_{k-1}|}{|x_{k-1}|} = |a| = a$,
zodat inderdaad de convergentatiefactor van het vast punt a is.

We beginnen de vastpuntiteratie met $a = 1/3$ in $x_0 = 1$ zoals ook in onderstaande figuur weergegeven. De fout ϵ komt overeen met de x-waarde na een iteratiestap (omdat het vast punt $x^* = 0$).

row	epsilon
0	1.00E+00
1	3.33E-01
2	1.11E-01
3	3.70E-02
4	1.23E-02
5	4.12E-03
6	1.37E-03
7	4.57E-04
8	1.52E-04
9	5.08E-05
10	1.69E-05

Figure 1: Tabel met epsilons per iteratiestap.

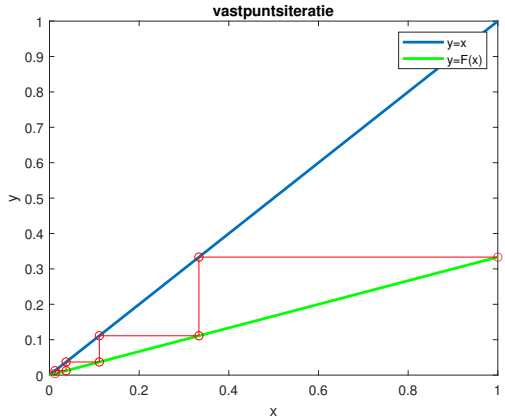

Figure 2: Visualisatie van de vastpuntsiteratie.

Opdracht 2 Als stopcriterium kozen wij $(|x_k - x_{k-1}| < tol) \wedge (|f(x_k)| < tol)$. Het eerste deel hiervan is rechtstreeks overgenomen van de Newton-Raphsonmethode. De andere voorwaarde zorgt ervoor dat de methode niet stopt in een lokaal minimum.

Omdat steeds $x = 1$ als startwaarde werd gebruikt, zorgde dit ervoor dat voor f_2 de methode in een lokaal minimum bleef vastzitten. Dit kan je ook zien op Figure 4, waarbij de fout van de methode ten opzichte van het werkelijk vast punt niet verbetert doorheen de iteraties en steeds oscilleert rond de functiewaarde in $x = 0$. Wanneer $x = 4$ als startwaarde werd gekozen, convergeerde de functie wel naar een nulpunt. Immers, in Figure 3 kan men zien dat de afgeleide van de functie in $x = 4$ inderdaad een zeer goede aanwijzing zal geven voor het nulpunt tussen $x = 4$ en $x = 6$.

We voerde de methode ook uit op de andere functies. We merkten op dat de andere functies wel convergeerden naar een nulpunt. Voor deze functies is de orde 2 (per keuze van de vastpuntsmethode Newton-Rhapson). De bijhorende convergentiefactoren staan genoteerd in Figure 5. De convergentie factor is $\sup_{k\in\mathbb{N}}\{\frac{\varepsilon_{k+1}}{\varepsilon_k}\}$, hetgeen voor een eindig aantal iteratiestappen gelijk is aan $\max_{k\in\{1,\ldots,N-1\}}\{\frac{\varepsilon_{k+1}}{\varepsilon_k}\}$.

Merk op (in Figure 3) dat $f_4(x)$ een meervoudig nulpunt heeft, noem dit x_0. De graad van het nulpunt blijkt drie te zijn (zie functievoorschrift van f_4). Dit heeft tot gevolg dat $\lim_{x\to x_0} f_4'(x) = 0$. Aangezien ook $\lim_{x\to x_0} f_4(x) = 0$, deelt de iteratiemethode een zeer klein getal door een ander zeer klein getal, hetgeen een onnauwkeurig resultaat geeft. Om dit op te lossen berekenen we $x_{k+1} = x_k - \frac{f_4(x_k)}{f_4'(x_k)}$ expliciet.
dit is zeer simpel te doen door middel van de productformule en de afgeleide van e^x en veeltermfuncties. dan volgt:

$$
\begin{aligned}
f_4(x) &= (8x^3 - 12x^2 + 6x - 1)e^x \\
f_4'(x) &= (8x^3 - 12x^2 + 6x - 1)e^x + (24x^2 - 24x + 6)e^x \\
&= (8x^3 + 12x^2 - 18x + 5)e^x
\end{aligned}
$$

vul hiermee $x_{k+1} = x_k - \frac{f_4(x_k)}{f_4'(x_k)}$. Dit geeft:

$$
\begin{aligned}
x_{k+1} &= x_k - \frac{(8x_k^3 - 12x_k^2 + 6x_k - 1)e^x}{(8x_k^3 + 12x_k^2 - 18x_k + 5)e^x} \\
&= \frac{8x_k^4 + 4x_k^3 - 6x_k^2 - x_k + 1}{8x_k^3 + 12x_k^2 - 18x_k + 5}
\end{aligned}
$$

Met deze expliciete iteratieformule verkrijgen we wel degelijk convergentie, met een fout zoals aangegeven op Figure 6.

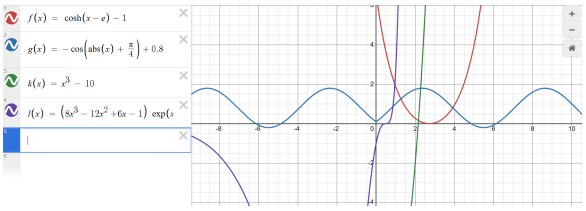

Figure 3: Grafiek van de functies $f(x) = f_1(x)$, $g(x) = f_2(x)$, $k(x) = f_3(x)$, $l(x) = f_4(x)$

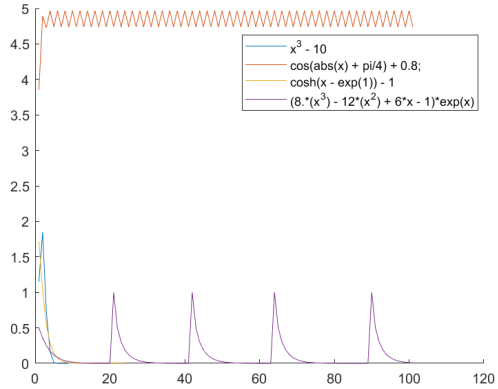

Figure 4: Grafiek van de fout van de methode ten opzichte van het werkelijk vast punt doorheen de iteraties.

functie f_i	convergentiefactor*
1	0
2	/
3	/
4	0.665314111499160

Figure 5: Tabel met convergentiefactoren.

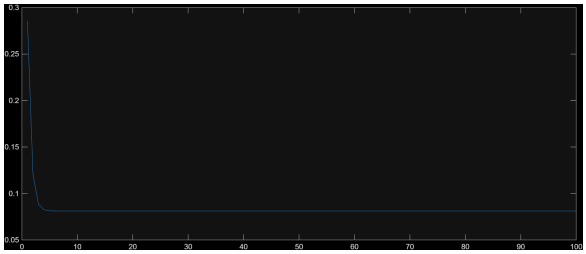

Figure 6: Convergentiegedrag van f_4 na het opstellen van een expliciete formule.

Opdracht 3 Voor de derde opdracht beschouwen we een complexe functie $f : \mathbb{C} \to \mathbb{C}$, waarbij we Newton-Raphson toepassen met verschillende startposities. Het script gaat alle punten in een rooster af waarbij de convergentiemethode telkens wordt toegepast met het coördinaat als startpunt. Indien er convergentie optreedt, slaan we op naar welk nulpunt (we slaan de index van het nulpunt op). Dit levert een matrix A op waarbij $A_{(x,y)}$ overeenkomt met de index van het nulpunt, startende vanuit positie $z = x + iy$. We kozen hierbij voor een rooster van -2 tot 2 voor beiden assen met telkens 1.000 onderverdelingen per as voor een totaal van 1.000.000 startposities. De resultaten hiervan zijn weergegeven in Figure 7 en Figure 8.
Men kan zien dat in de nabije omgeving rond een nulpunt, een uniforme kleur aanwezig is. Dat is logisch. Immers, als er convergentie optreedt, zullen alle punten in de omgeving convergeren naar het nulpunt (per definitie van convergentie). Daarnaast zijn er ook prachtige patronen waarneembaar daar waar een punt ongeveer even dicht bij minstens twee nulpunten ligt. We vermoeden dat deze patronen zich voordoen doordat we discrete stappen zetten op een plek waar de Newton-Raphson methode zeer gevoelig is. De iteratiemethode zet dan een substantiele stap richting een nulpunt dat niet het dichtstbijzijnde nulpunt is. Eens deze stap gezet, convergeert het (wel) naar het dichstbijzijnde nulpunt. De grafiek demonstreert dus in zekere zin de gevoeligheid van de vastpuntsmethode op instabiele punten.
Het uitvoeren van dit programma duurt best lang indien men een goede resolutie wilt, t.t.z. een hoge dimensie van de matrix A. Dat komt doordat voor elke rij in de matrix, alle kolommen afgegaan worden waarbij telkens Newton-Raphson wordt uitgevoerd. Het aantal rijen en kolommen van deze matrix wordt bepaalt door Ny en Nx respectievelijk. De convergentiemethode zelf heeft een tijdscomplexiteit van $\mathcal{O}(max_iter)$. Bijgevolg worden er dus $\mathcal{O}(Ny \cdot Nx \cdot max_iter)$ primaire operaties uitgevoerd. Dit is ook meteen de duurste operatie, Aangezien alle andere operaties in het algoritme strikt goedkoper zijn. Zo ook bijvoorbeeld het zoeken van het dichtstbijzijnd nulpunt, hetgeen lineair in het aantal nulpunten van de functie berekent kan worden. Voor een vaste functie is het aantal punten uiteraard constant.

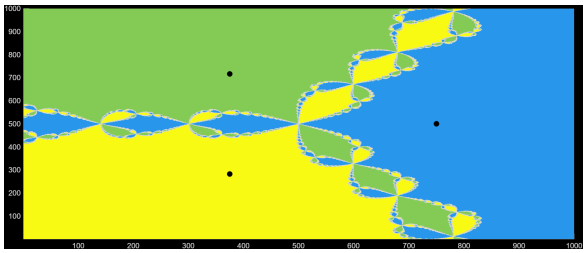

Figure 7: Grafiek van de matrix A voor de functie $f : \mathbb{C} \to \mathbb{C}, x \mapsto x^3 - 1$.

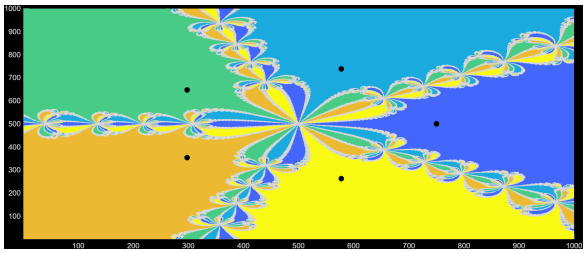

Figure 8: Grafiek van de matrix A voor de functie $f : \mathbb{C} \to \mathbb{C}, x \mapsto x^5 - 1$

Opdracht 4 Om het effect van de multipliciteit van een nulpunt op het Newtonbassin te zien, stellen we de functie $(z-1)^m(z+1)$ op. Hierin stelt de variabele m de multipliciteit van het nulpunt $z = 1$ voor. Figuur 9 toont de Newtonbassins voor verschillende multipliciteiten, weergegeven in het domein waarbij zowel x als y variëren van -2 tot 2. Het bassin visualizeren we door de punten $z \in \mathbb{C}$ waarvoor de iteratie convergeert naar het nulpunt in maximaal 60 iteraties blauw te kleuren.
In de figuren is te zien hoe de bassins een fractaalstructuur vormen. Deze fractaal wordt ook steeds smaller in functie van m en neemt zo een kleiner percentage van het bekeken gebied in beslag. Dit betekent dat er meer startwaarden van de Newton-Raphsonmethode naar het nulpunt convergeren met de hogere multipliciteit.
Een ander gevolg van hogere multipliciteit van een nulpunt is dat de Newton-Raphsonmethode trager convergeert. Een hogere graad van het nulpunt van de veelterm zorgt er namelijk voor dat de methode lineair convergeert in plaats van kwadratisch. Dit kan voor sommige startwaarde een significant effect hebben op de convergentiesnelheid. Figure 10 geeft het aantal iteratiestappen weer die nodig zijn om tot de nulpunten te convergeren. Hierop is duidelijk te zien dat de multipliciteit een effect heeft op de convergentiesnelheid.

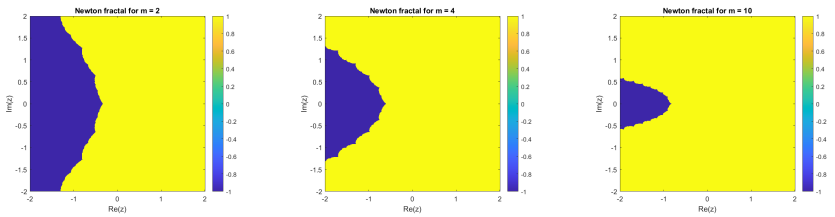

Figure 9: newtonbassin van $(z-1)^m(z+1)$ voor verscheidene m

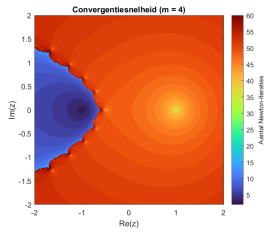

Figure 10: aantal iteraties tot nulpunt

Opdracht 5 Eerst berekenden we de error ϵ tussen de eindige differentie $f'(x) = \frac{f(x_0+h)-f(x_0-h)}{2h}$ en de exacte afgeleide. Dat hebben we geplot op logaritmische schaal in Figure 9.
Men kan twee fenomenen zien op de grafiek. Enerzijds, wanneer we de afstand h te groot maken, dan zijn we de grafiek te sterk aan het benaderen; we nemen punten die te ver van elkaar liggen. Anderzijds, wanneer we de afstand h te klein maken, dan rekent matlab met zo'n kleine getallen dat afrondingsfouten te sterk doorwegen, zodat de accuraatheid net kleiner wordt.
We constateren op basis van de figuur dat het praktisch optimum iets groter blijkt te zijn dan het theoretisch minimum 10^{-8}.
Vervolgens hermaakten we het fractaal van opdracht 3, maar ditmaal met de nieuwe eindige differentie. Op kleine schaal zullen er kleine onnauwkeurigheden optreden die zorgen voor een (lichtjes) afwijkend fractaal. Echter, met het blote oog ziet men geen verschil tussen de grafiek op basis van de analytische afgeleide en die op basis van de numeriek benaderde afgeleide.

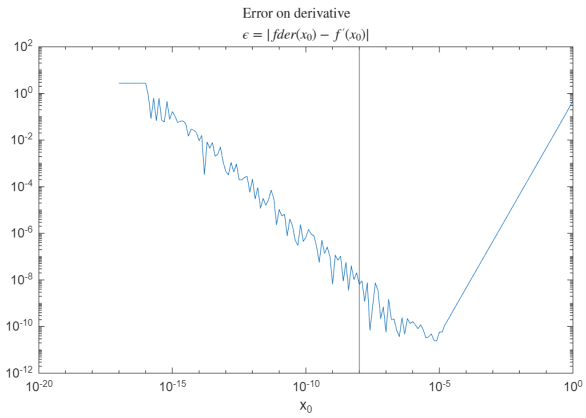

Figure 11: Grafiek van de error op de benadering van de afgeleide. In rood 10^{-8}, het theoretisch optimum.

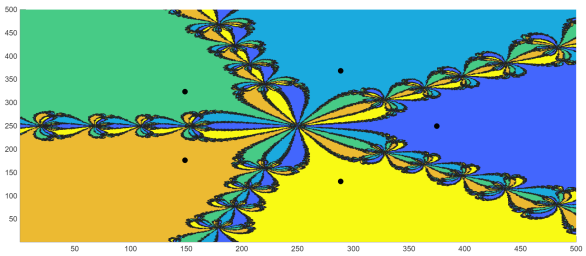

Figure 12: Grafiek van de matrix A met eindige differentie voor $z^3 - 1$.